

Fair Intelligence CMS & Role Modelling Plan

Oct 3, 2026 · @Refal

Summary

Fair Intelligence now runs three roles (PR, Brand & KOL, Social Media) on any workspace. Today each role is a constant in code (`src/roles/model.ts`). That constant sets the home page, sidebar, dashboard, CeMO's voice, suggested questions, deck templates, skill order, alert rule and the things it watches. Changing any of it means a code change and a deploy. Onboarding a workspace is the same: an ETL contract in a JSON file, a Python script run by an engineer, and settings edited in the database.

This plan turns both into a product.

- **The CMS is where workspaces are born and run.** An admin creates a workspace, uploads a dump (later, connects the daily sync), maps brands and handles, tunes relevance terms against live examples, sets topics, chooses which roles the workspace offers, invites people, and watches data health. No engineer and no script.
- **Every workspace gets its own model of each role.** The product team publishes a *core* model per role (PR, Brand & KOL, Social Media). Each workspace runs a *copy* of it that its own people reshape: which dashboard tiles show and in what order, alert thresholds, deck templates, CeMO's house rules, the words it uses, the competitors it watches. Each person can then tune their own view on top. The effective model is $\text{core} \oplus \text{workspace} \oplus \text{person}$, resolved at request time.
- **Every change is recorded, previewable and reversible.** Edits create versions. A change that alters a number (relevance terms, brand grouping, thresholds) shows a before/after against real data before it goes live, and can be rolled back in one click.
- **The product learns from all of it.** Every reshape and every use (what people open, pin, ask, export, ignore, override) becomes a signal. Signals roll up across workspaces into insights for the product team: "7 of 9 PR workspaces raised the crisis floor from 50 to 100 negative comments". The team turns an insight into a proposal, publishes it as a new core version, and workspaces that follow the latest pick it up, while workspaces that pinned a version keep theirs. That is the "ultimate role model": each role gets sharper the more teams use it, and no workspace loses what it has shaped.

Two rules hold throughout, both from CLAUDE.md. Nothing is shared across workspaces: learning moves *shapes* (settings, layouts, thresholds), never data, text or numbers from one workspace into another. And the model never computes numbers: a role model can change what is shown and how CeMO speaks, but every figure still comes from SQL in skills, the dashboard and the deck facts.

Delivery is six phases over roughly ten to twelve weeks of build, starting with the foundation that moves the three role constants into versioned tables with no visible change (section 13). Section 14 lists the decisions needed from you before phase 1 starts.

The three layers of a role model

A role model is how the product behaves for one kind of team. A workspace is the data that team looks at. The two meet in a *team* (workspace × role), and that is where the effective model lives.

Layer 1 — Core model (owned by the product team)

One per role: PR, Brand & KOL, Social Media, and any role added later (for example Customer Care or Performance Marketing). It is the product team's best current answer to "what does a PR team need". It is versioned (`pr@1` , `pr@2` , ...) and published from Model Studio (section 10). Each field carries a **policy** that says who may change it below:

- **Locked** — only the product team. Examples: the definition of "measured" for Social medians, the ladder's three levels and their order, the rule that issue labels compare shares not counts, the minimum comment floor below which no status is shown.
- **Workspace-editable** — a workspace admin may override it for everyone in that workspace. Examples: the crisis multiplier, the negative-comment floor, which tiles show, deck templates on offer, CeMO's house rules, brand nicknames.
- **Person-editable** — each member may tune it for themselves. Examples: default window (7/14/30 days), brand in focus, tile order, which alerts reach their inbox, CeMO answer length.

Layer 2 — Workspace model (owned by the workspace's admins)

One per workspace × role. It stores only the *differences* from the core version it is based on, never a full copy, so a new core version can flow through everything the workspace did not touch. It records:

- `base_version` — the core version it sits on.
- `follow` — **latest** (move to new core versions automatically, after a preview window) or **pinned** (stay on `base_version` until an admin upgrades).
- `overrides` — a sparse set of field paths and values, for example `alert.crisis_multiplier = 2.5` , `dashboard.tiles = [...]` , `voice.house_rules += "Call GoPay 'kami' when drafting holding lines"` .
- `memory` — durable facts the team has taught CeMO about itself ("our fiscal month starts on the 26th", "Paylater launches 1 Nov"). Memory is workspace data, never shared.

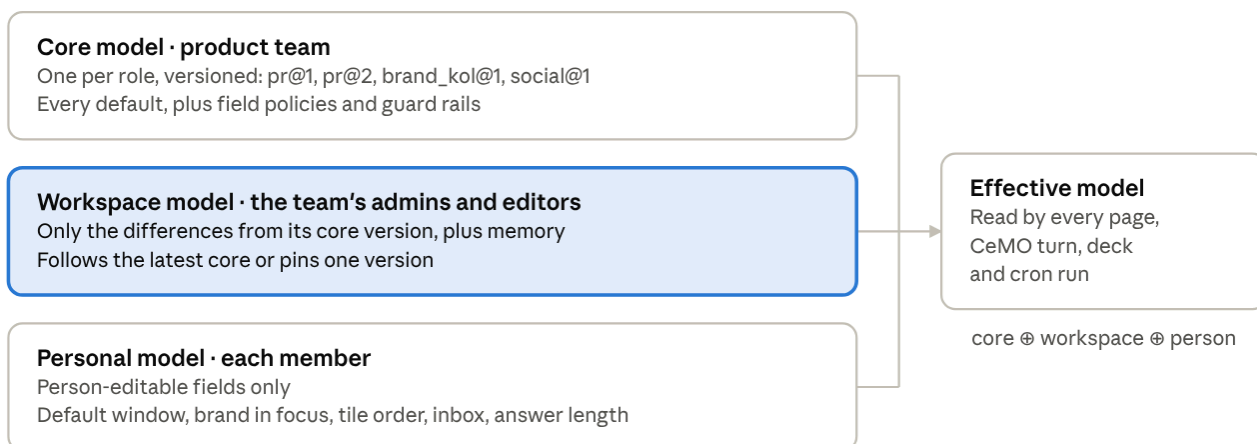
Layer 3 — Personal model (owned by each person)

One per person × workspace × role. Same sparse shape, restricted to person-editable fields. It lets a head of PR and a junior analyst share a workspace model but land on different default views.

Resolution

At request time the server resolves `effective = core@base_version ⊕ workspace.overrides ⊕ person.overrides`, field by field, dropping any override the field's policy does not allow at that layer. Resolution is pure and cached per request, so every page, CeMO turn, deck generation and cron run sees the same model. The current code path becomes a fallback: if the tables are empty, the constants in `src/roles/model.ts` are the core v1.

Each team runs three layers, resolved on every request



A lower layer overrides a field only where that field's policy allows it

role model layers · core, workspace, person

The workspace layer is the new part: it holds only what a team changed, so a new core version still reaches everything the team left alone.

Two things are deliberately **not** in a role model: data (brands, handles, relevance terms, topics belong to the workspace, section 8) and numbers (thresholds are model fields, but computing against them stays in SQL).

What a role model contains

Today a `RoleModel` has fifteen fields: identity (label, short, description, tone), navigation (home, nav), CeMO (voice, hero title and intro, suggested questions), decks (template ids), the composer's skill order, and two blocks of thresholds (`alert` for PR, `watch` for

Social). The CMS keeps all of them and organises the model into nine parts, so that every part a user can see is also a part they can shape.

Part	What it decides	Default policy
Identity	Name, short name, description, colour, icon shown in the team picker and sidebar	Locked (workspace may rename)
Navigation	Home page and sidebar entries, from the fixed set Dashboard, Weekly Reports, Decks, Pulse, Chats, Reports	Locked set, workspace may hide or reorder
Dashboard	Which tiles show, their order, default window, default brand in focus, default platform, which tiles a person may hide	Workspace-editable; window, focus and order person-editable
Signals	Thresholds that move a status or flag a post: PR's ladder (watch / issue / crisis multiples, comment floor, baseline days, recovery days) and Social's watch rules (underperform %, comment-storm floor, minimum posts per account)	Ladder shape locked; numbers workspace-editable within guard rails
Decks	Templates offered, their order, default slides per template, default grain and cadence, cover wording	Workspace-editable
CeMO	Voice (who it is on this team), house rules, what it may draft (holding lines, Q&A, captions, calendars), what it must refuse or redirect, empty-screen title and intro, suggested questions	Voice core locked, house rules and suggestions workspace-editable, answer length person-editable
Skills	Which analyses the composer shows first, which are hidden on this team, plain titles and example questions	Workspace-editable
Vocabulary	Words the product uses: "issue" vs "topic", "creator" vs "KOL", "own accounts" vs "brand accounts", the client's nickname, Bahasa or English labels	Workspace-editable

Part	What it decides	Default policy
Delivery	Scheduled reports and alerts: who gets what, by email or not, quiet hours, digest day	Workspace-editable, inbox choices person-editable

Each part is a typed block in the model's `spec` (section 12). Guard rails are part of the core model: a PR workspace may set the crisis multiple between 2× and 6× but not below the issue multiple, and the comment floor cannot drop under 20, because below that a status is noise.

The three roles today, as models

PR (`pr@1` , coral). Home is the reputation dashboard: status ladder, share of voice, sentiment and net sentiment by brand, issues labelled *only us / whole category / mixed*, amplifiers, service complaints separated from reputation stories. Signals: watch at 1.5× the 28-day negative baseline, issue at 2×, crisis at 3× with at least 50 negative comments, a 50-comment floor, recovering within 7 days. Decks: reputation weekly, reputation monthly, issue post-mortem. CeMO is the comms advisor: drafts holding statements, spokesperson Q&A, replies and escalation notes with confirmable facts in [brackets], sends service complaints to customer care, refuses to advise on pile-ons. Skills shown first: comments, conversation, posts.

Brand & KOL (`brand_kol@1` , blue). Home is the competitor dashboard: scoreboard by platform, landscape (tiers, products, posting, close-ups), creators, campaigns. Signals: tier and affiliate bands from `src/config` (these stay workspace data, see section 8). Decks: the competitor templates and the Weekly Competitor Pulse. CeMO is the category analyst; its voice is empty today and inherits the workspace persona. This is the model that most needs a written voice; phase 2 adds one.

Social Media (`social@1` , mint). Home is the own-accounts dashboard: posts per account, engagement as the measure, views only where reported, formats, posting times, community replies, own vs competitors. Signals: a post underperforms at 50% below its account's median, a comment storm at 20 negative, an account needs 5 measured posts before it is judged. Decks: content monthly, content teardown. CeMO is the content lead: drafts captions, calendars, replies and briefs.

Example of a workspace reshaping its model

GoPay's PR team on `fintech-id` , after a month of use, might hold these overrides on `pr@1` :

- `signals.crisis_multiple = 4` and `signals.min_negative = 100` (fintech has louder baselines than beauty).

- `dashboard.tiles` without *Amplifiers* and with *Service complaints* moved to the top.
- `cemo.house_rules` : "Never draft anything about OJK or Bank Indonesia without saying Legal must review." and "Refer to the client as GoPay, never Gojek."
- `vocabulary.issue = "isu"` , `vocabulary.client = "GoPay"` .
- `decks.templates = ["reputation-weekly", "issue-postmortem"]` , weekly delivered Monday 08:00 WIB to the head of corporate comms.

None of those touch the core. If the product team ships `pr@2` with a new *Regulator mentions* tile, GoPay gets it (if it follows latest) without losing any of the above.

How users reshape their model, inside the product

Most reshaping should not happen in the CMS. It should happen where people already work, in small gestures, and the product records each one as a change to the workspace or personal model. The CMS (Model Studio, section 10) is where admins see the whole model, tidy it and approve bigger changes.

Every gesture below has three things: **where** it happens, **which layer** it writes to, and **who** may do it. A member's gesture writes to their personal model. An editor or admin is asked once: "Just for you, or for the whole GoPay PR team?"

Dashboard

- **Hide, move, resize a tile.** A small "Customise" toggle at the top of the dashboard turns tiles into cards with a handle and an eye. Writes `dashboard.tiles` .
- **Set defaults.** When someone changes the window, brand or platform and keeps it for a session, the filter row offers "Make this my default". Writes `dashboard.defaults` (personal).
- **Add a tile from Chats.** An answer with a chart already offers "Show chart"; it gains "Add to dashboard", which stores the analysis as a tile that re-runs on load (the same mechanism as Pulse cards from a `skill_run_id`). Writes `dashboard.custom_tiles` (workspace if an editor chooses so).
- **Rename a tile** ("Share of voice" → "SOV vs e-wallets"). Writes `vocabulary` .

Signals and alerts

- **Tune a threshold where it fires.** The status ladder card gets a "Why this level?" drawer showing the rule in words ("Issue: negative comments at 2.1× the 28-day usual, 340 today"). Admins can move the slider there, with a preview of how many days in the last 90 would have been watch / issue / crisis under the new rule (computed in SQL over history, section 11).

- **Mute or snooze.** "Not an issue" on an issue marks it known for 7 days; three "not an issue" on the same theme within a month becomes a suggestion to the admin ("Mute theme Promo for status? It was dismissed 3 times").
- **Choose delivery.** Each person picks which alerts reach their inbox and quiet hours. Personal.

Decks

- **Save as template.** After editing a deck's slides and wording, "Save as a template for this team" stores it as a workspace template with its slide list and defaults. Writes `decks.templates`.
- **Retire a template.** Hiding a template the team never uses. Workspace.
- **Cover and sign-off wording.** Edited once per workspace.

CeMO

- **House rules.** A "CeMO rules" panel in Chats settings lists the workspace's rules in plain sentences, each with who added it and when. A member can propose one; an editor approves.
- **Teach from a correction.** When someone pushes back ("No, we call it isu, not issue" or "Don't count Shopee as a competitor here") CeMO answers in one plain sentence and, if the correction is durable, offers "Remember this for the team?". Accepted, it is stored as a house rule or a memory fact. Corrections about *data* (Shopee is not a competitor) are routed to the workspace's brand set (section 8), not to CeMO's prompt, because a prompt cannot change a number.
- **Memory.** Facts about the client (launch dates, spokespeople, campaign names, what "last month" means for them) live in `workspace_memory`, shown and editable in the same panel. CeMO reads them every turn.
- **Suggested questions.** Editors replace or add to the four suggestions on the empty screen.
- **Answer style.** Personal: shorter or fuller answers, Bahasa or English by default.

Skills

- **Pin and hide in the "/" menu.** Hiding an analysis for the team removes it from the composer's menu, not from what CeMO may run when asked.

What every gesture shares

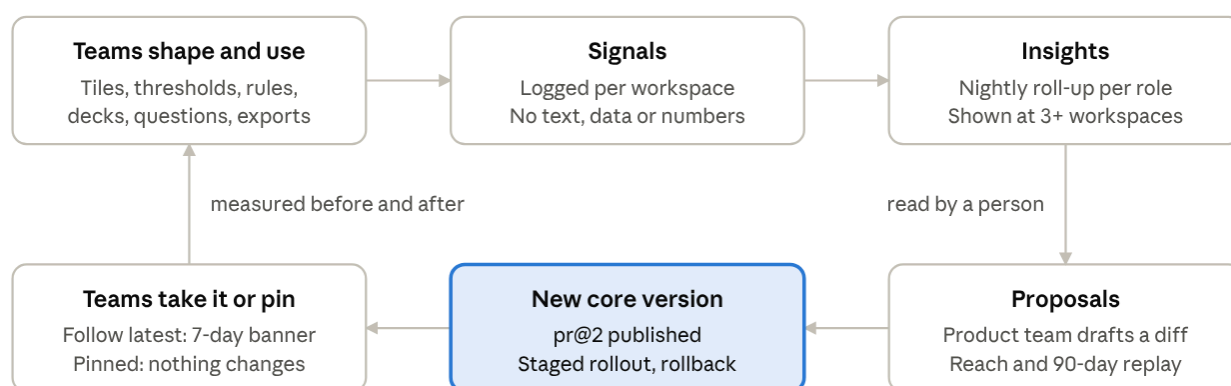
- It is recorded as a **model change** with author, time, layer, field path, old value and new value (section 11).

- It is **reversible**: a "Changes" list on each surface shows the last ten changes with Undo, and Model Studio shows the full history.
- It is **visible**: when a workspace override hides something the core model shows, the surface says so in small text ("Amplifiers hidden for this team · Show").
- It emits a **signal** (section 5), whether the change was kept or undone.

The learning loop: how roles get better the more teams use them

This is what makes the model "ultimate". Each workspace shapes its own copy; the product team sees, across all of them, what people keep, change, use and ignore, and folds the patterns that hold into the next core version. The loop has five steps.

Each role gets better as more teams use and reshape it



learning loop · 6 steps

Only settings and counts travel around this loop; data and text stay in their workspace (the boundary is stated at the end of this section).

1. Signals

A signal is one small, structured event, written to `model_events` with workspace, role, person, model version, surface, kind and a small payload. There are two families.

Shaping signals come from section 4: a tile hidden, a threshold moved, a template saved, a house rule added, a suggestion replaced, a skill hidden, and every Undo of those.

Use signals come from normal work:

Surface	Signals
Dashboard	tile viewed (in viewport $\geq$ 2 s), "Ask why" clicked, filter changed, time on page

Surface	Signals
Signals	status changed, alert sent, alert opened, "not an issue" pressed, ladder drawer opened
Decks	template chosen, slide removed or reordered, wording edited (which slide, not the text), exported, emailed, version made by cron and opened or not
Chats	analysis run (skill and layer), clarifying question asked, pushback, "Show chart", evidence pane opened, export, "Turn into a report / deck", answer copied, conversation abandoned after one turn
CeMO drafts	draft requested (holding line, Q&A, caption), kept, edited heavily, discarded

What a signal never carries: the text of a question or answer, a comment, a post, a brand name from another workspace, or a number from the data. Questions are reduced to the *analysis that answered them* and a coarse intent label computed inside the workspace ("issue check", "competitor comparison", "draft"). This is the privacy boundary, and it is what lets learning cross workspaces without anything else crossing.

2. Insights

A nightly job (Vercel Cron, section 12) rolls signals up per role and core version into `model_insights`. Each insight is one sentence with its counts and the workspaces behind it, for example:

- "Crisis multiple raised from 3× in 6 of 9 PR workspaces; median chosen value 4×."
- "Amplifiers tile hidden by 5 of 9 PR workspaces within two weeks; viewed under 2 s in the others."
- "Brand & KOL: 41% of conversations end in a Top content run; Top content is 6th in the composer."
- "Social: content-teardown deck is made and never opened in 70% of monthly runs."
- "PR drafts: holding statements kept 3 in 10 times; the most edited part is the closing line."

Insights only count; a person on the product team reads them. A minimum of three workspaces is required before an insight about a setting is shown, so no single client's choices are visible as such.

3. Proposals

The product team turns an insight into a **proposal**: a diff against the current core ("pr@2 : crisis multiple 3× → 4×; Amplifiers tile off by default; Service complaints tile moved up"). Model Studio shows the proposal's reach before it is published: how many workspaces follow latest, how many have already overridden each field (and therefore will not feel it), and for any number-changing field, a replay of the last 90 days per workspace under the new value ("GoPay: 4 days fewer at crisis"). The replay is SQL over history, never an estimate.

Proposals can also come from a workspace upward. An admin who has shaped something well can press "Suggest to Fair" on any override; it appears in Model Studio with its workspace name visible only to Fair staff.

4. Versions and rollout

Publishing a proposal creates a new core version. Rollout respects each workspace's **follow** choice:

- **Follow latest**: the workspace sees a banner for 7 days ("PR model updated: Service complaints moved up, crisis now 4×. Review · Keep old"), then the new version applies. Its overrides are re-applied on top; an override that no longer fits the new schema is kept aside and reported, never silently dropped.
- **Pinned**: nothing changes. Model Studio shows "2 versions behind" and what each version would change.

A core version can be rolled back by the product team; workspaces on it return to the previous version with their overrides intact.

5. Measure

Each version records the signals that motivated it. Two to four weeks after rollout, Model Studio compares the same signals before and after for workspaces that took the version, against workspaces that stayed pinned (a natural control). "After pr@2, alerts opened rose from 31% to 54%; pinned workspaces unchanged." That closes the loop and is the evidence for the next proposal.

The boundary, stated once

Across workspaces moves only: which fields were changed, to which setting values (thresholds, tile ids, template ids, skill ids), and counts of use. Never: data rows, text people typed, CeMO's answers, house-rule wording, memory facts, brand names, or numbers computed from a workspace's data. House-rule *wording* stays in its workspace;

Model Studio sees only that a rule was added to the "drafting" or "naming" category. A client's contract can switch its workspace out of learning entirely (`settings.learning = false`); its signals are then kept for its own Model Studio view only.

The CMS: who uses it and what it holds

Who uses it

Person	Where they sit	What they do in the CMS
Product team (Fair)	Fair staff, all workspaces	Owns core role models: reads insights, writes proposals, publishes and rolls back versions, sets guard rails and field policies
Data ops (Fair)	Fair staff, assigned workspaces	Creates workspaces, uploads dumps and later connects the sync, maps brands and handles, tunes relevance and topics, watches health
Account lead (Fair)	Fair staff, their clients' workspaces	Sets which roles a workspace offers, invites the client's admins, reviews the workspace's model before go-live, sees its usage
Workspace admin (client)	One or more workspaces	Invites and removes people, grants roles, edits the workspace model, approves house rules, sets the client brand and delivery
Editor (client)	A workspace and role	Shapes the team's model from inside the product (section 4); proposes rules; cannot change people or data
Member (client)	A workspace and role	Uses the product; shapes only their personal model

Today there are two levels (`users.role` is owner or member), a user row exists per workspace, and only owners can switch between workspaces. The CMS replaces that with memberships (section 9).

The areas

The CMS lives at `/admin` inside the same Next.js app, behind the same sign-in, with its own sidebar. Client admins see only the areas and workspaces they hold; Fair staff see more. Nothing in `/admin` is reachable over the MCP connector.

1. **Workspaces** — list with kind (category panel, profile, listening), status (draft, loading, review, live, paused, archived), roles offered, client brand, last load, health. Creating a workspace starts the wizard (section 7).
2. **Data** — per workspace: sources, loads with their reports, schedule of the daily sync, re-run and roll back a load. Grows out of today's `/data` page and `data_loads` table.
3. **Brands & relevance** — brand set, handles per platform, owned accounts, groupings ("BCA and blu by BCA are one brand"), relevance terms with a live preview (section 8).
4. **Topics** — the workspace's taxonomy, its catch-all, examples per topic, the share each holds, merges and splits (section 8).
5. **Schema modules** — which optional data a workspace carries: core listening, beauty commerce (product type, content format, affiliate bands), creator tiers, caption reading (section 8).
6. **People** — members, invites, roles per person, permission levels, sign-in method, last seen (section 9).
7. **Model Studio** — the effective model per team, every override with its author, versions, preview, publish, rollback (section 10). For Fair staff it also holds the core models, insights and proposals.
8. **Health** — data freshness per platform, rows per day, relevance rate, share in catch-all topic, labelling backlog, failed crons, unusual bursts ("@gopayindonesia: 57–65 own Instagram posts a day on 7–9 Sep, usually 2–4").
9. **Audit** — every change in the CMS and every model change from inside the product, filterable, exportable (section 11).
10. **Usage** — per workspace and team: active people, conversations, decks made and opened, alerts opened. Client admins see their own; Fair sees all, plus the cross-workspace Learning view in Model Studio.

Design principles

- **Preview before apply.** Anything that changes what is counted shows the before and after on real data first.
- **Plain words.** The same language as the product: "Brand set", "Counts as about GoPay", "Team", not "entity", "filter predicate", "tenant".

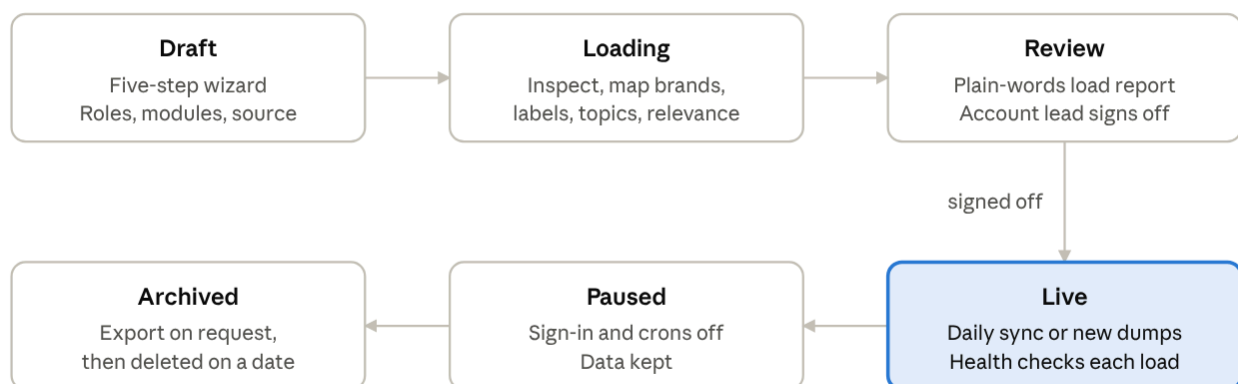
- **One place per fact.** A brand's handles live in Brands, not in a JSON contract and a settings blob and a seed CSV. The JSON contracts in `etl/listening/` and `etl/profiles/` become exports of CMS state, kept for reproducibility.
- **Scripts keep working.** `etl/load_listening.py` and `etl/load_profile.py` stay callable by engineers; the CMS calls the same steps.

Workspace lifecycle

Onboarding Fintech took one engineer most of a day: reading the dump, asking you how to group brands and map sentiment, writing `etl/listening/fintech-id.json`, running the loader, finding that 27% of posts did not name their brand, adding the relevance flag, checking screens. Each of those steps becomes a screen in the CMS, so data ops can onboard a workspace in an hour and the next workspace (a bank, a telco, a beauty brand on listening) needs no code.

A workspace moves through six states: **draft** → **loading** → **review** → **live** → **paused** → **archived**.

A workspace goes live only after a person signs off its load report



workspace lifecycle · 6 states

Clients only ever see a live workspace; review is where a person reads the load report before anyone else does.

1. Create (draft)

A five-step wizard:

1. **Basics** — name, id (slug, fixed once live), kind (category panel, profile, listening), category (beauty, fintech, telco, FMCG, other; drives which lexicons and schema modules apply), language, time zone (Asia/Jakarta default).

2. **Client** — the client brand, if any (today `workspaces.client_brand_id`), and the client's display name.
3. **Roles** — which teams the workspace offers (PR, Brand & KOL, Social Media) and, for each, which core version it starts on and whether it follows latest. Defaults by kind: profile → PR, panel → Brand & KOL, listening → all three.
4. **Modules** — core listening always; beauty commerce, creator tiers and caption reading switched on by category (section 8).
5. **Source** — upload a dump (zip) now, or connect the daily sync once it exists.

2. Load (loading)

The dump is stored, then read by a job (section 12). The loader is today's Python, split into steps the CMS can show and stop between:

- **Inspect** — files found, platforms, date range, row counts, columns recognised and unrecognised. For Fintech: 13,275 posts, 116,366 comments, 114,633 snapshots, X/Threads/TikTok/Instagram/YouTube.
- **Map brands** — handles found in the dump, grouped by suggestion (same name across platforms, known aliases), for data ops to confirm, merge ("BCA + blu by BCA") or drop. Owned accounts ticked here.
- **Map labels** — the dump's sentiment scale to the product's three classes ("Average → negative" for Fintech), with counts per mapping so the effect is visible.
- **Topics** — the dump's topics imported as the workspace taxonomy, with share per topic; anything over 30% in a catch-all is flagged.
- **Relevance** — terms proposed from brand names and handles, with the share of posts that would count and examples on each side (section 8).
- **Write** — rows inserted with `workspace_id`, views refreshed.

3. Review

A **load report** in plain words, the same one kept for every later load:

- Totals by platform and day, with thin days flagged (the last days of a dump are often incomplete; the dashboards already end windows on the last settled day).
- Relevance: 3,548 of 13,275 posts (27%) do not name their brand; Threads is about half off-topic.
- Topics: "Others" holds 42–59% of comments; Paylater has 10 comments.
- Bursts and gaps: own Instagram posts for @gopayindonesia jump to 57–65 a day on 7–9 Sep and stop after 9 Sep.
- Missing data: follower history has one capture per account, so follower growth is shown as unavailable.

- A screenshot strip of each team's dashboard on this data, so the account lead sees what the client will see.

The account lead signs off; the workspace goes live. Problems become **notes** on the workspace that CeMO can read ("Own Instagram posts are not captured after 9 Sep"), so it does not draw conclusions from a gap.

4. Live

- **Daily sync** (when the team's read-only sync exists) runs as a scheduled load; each run gets a load report. Until then, new dumps are uploaded and loaded as increments, de-duplicated on (platform, url, brand).
- **Health** checks run after each load and nightly: freshness per platform, rows per day vs the 28-day usual, relevance rate, catch-all share, unlabelled comments backlog, crons that failed. A check that fails twice raises a note to data ops and, if it affects numbers on screen, a quiet banner in the product ("TikTok data is 3 days behind").
- **Re-load and roll back** — every load is a batch with an id; a bad load can be removed and the views refreshed.

5. Paused

Sign-in disabled for client members, crons and scheduled decks stopped, data kept. Used between contracts.

6. Archived

Data exported to a file for the client on request, then deleted on a set date; the workspace's model overrides and memory are deleted with it. Its anonymous signals (section 5) remain in the learning roll-ups, because they no longer identify it.

Brands, relevance, topics and schema modules

These are workspace **data settings**, not role model fields: they decide what is counted, so every role on the workspace sees the same result. Changing one is a number-changing edit and goes through preview and approval (section 11).

Brands

- **Brand set** — canonical slug, display name, nickname CeMO may use, logo, colour, whether it is the client, whether it is shown on dashboards (a brand can be loaded but kept off the competitor set).

- **Handles** — per platform, many per brand; each marked owned or not. Owned vs earned then works on every platform the workspace holds, as it already does for listening workspaces.
- **Groups** — merges like "BCA + blu by BCA" or "Shopee + ShopeePay" (kept separate in Fintech today). A merge is reversible because it maps handles, not rows.
- **Competitor sets** — named subsets ("E-wallets", "Banks") that teams can pick as a filter and that decks can take as their brand list.

Relevance

Today a listening post counts when it is from an owned account, tags the brand, or its caption matches one of the brand's terms (a term matches at the start of a word; an all-caps term is case-sensitive and must be a whole word). That rule is right for any workspace; what differs is the terms. The CMS makes them a tool:

- **Terms editor** per brand, with three lists: *counts* ("gopay", "GoPay Later"), *only with* ("OVO" counts only as a whole capitalised word), *never* ("go pay attention").
- **Live preview** — as terms change, the panel shows the share of the brand's posts that would count, by platform, with ten examples flipping in each direction and their views. Fintech today: Threads is about 50% off-topic; X mostly on-topic.
- **Apply** recomputes `posts.relevant` for that brand in a job and refreshes the views. The before/after (posts, views, share of voice) is stored with the change.
- **Suggestions** — handles and words that often co-occur with the brand's tagged posts are offered as candidate terms; a person always confirms.

This answers your first decision from the PR plan: off-topic filtering works for any onboarded workspace, because the rule is general and the terms are workspace data edited here, not code.

Topics

Topics arrive with the dump today (Fintech: Others, Promo, Service, Paylater and more, with "Others" holding 42–59% of comments and Paylater only 10). You said topics are adjustable per workspace; the CMS makes that concrete:

- **Taxonomy editor** — rename, merge, split, reorder (`topics.sort_order`), mark the catch-all (`is_catch_all`), give each topic a one-line definition and three example comments.
- **Health** — share per topic over time, with a warning when the catch-all passes 30% or a topic falls under 30 comments a month.
- **Mapping** — when the listening team changes a topic upstream, a mapping table keeps the workspace's names stable.

- **Role use** — each topic can be tagged *service, promo, product, reputation*; PR's service list and "only us / whole category" logic read those tags instead of the theme matching in code today.

Re-labelling the catch-all with our own model is possible later (`/api/cron/label` already labels profile comments) but is a separate decision, listed in section 14.

Schema modules

The complete listening schema is the template for every workspace (DECISIONS, 3 Oct 2026). On top of it, optional modules switch on extra data and the screens that use it:

Module	Adds	Used by
Core listening (always)	posts, comments with sentiment, topics, snapshots day 0–30, owned vs earned, relevance	all roles
Beauty commerce	product type, content format, affiliate signals, product lexicon	Brand & KOL dashboards, landscape, products
Creator tiers	follower bands from <code>src/config</code> per category	Brand & KOL, Social comparisons
Caption reading	product, event, offer, hook, angle per post (<code>posts.cap_*</code>), floor and start date	decks, query builder
Comment labelling	our own three-class sentiment where the source has none	profile workspaces

Each module declares the columns it needs, its config (bands, floors), and which skills and slides depend on it. A skill whose module is off returns "unavailable" with a plain message, as today. Tier and affiliate bands stay in `src/config` as defaults and become per-category overrides in the CMS, still read by SQL, never hardcoded.

People and access

What changes

Today a person is a row in `users` per workspace (unique on workspace + email), with `role` owner or member; owners may switch workspace, the team (role) is a cookie (`fi_role`) any member can change among the workspace's roles. That was enough for three workspaces run by Fair. It is not enough once clients run their own.

The CMS splits identity from access:

- **Accounts** — one per person (email, name, password hash or SSO link, WhatsApp). One sign-in reaches every workspace the person belongs to.
- **Memberships** — account × workspace, with a permission level and the roles (teams) the person may use there. A PR agency person might hold GoPay (PR) and a bank (PR); a GoPay brand manager might hold GoPay (Brand & KOL, Social).
- **Fair staff** — a flag on the account plus an assignment list, so data ops and account leads see only their workspaces; the product team sees all.

Permission levels

Level	Use the product	Shape own model	Shape team model	Approve rules	People	Data settings
Member	✓	✓	—	—	—	—
Editor	✓	✓	✓ (non-number fields)	✓	—	—
Admin	✓	✓	✓ (all workspace fields)	✓	✓	view
Fair data ops	✓	✓	✓	✓	view	✓
Fair product	✓	✓	✓	✓	✓	✓, plus core models

Permissions are checked on the server in one function (`can(account, workspace, action)`), used by pages, routes, the CMS and the model-change API alike.

Teams and sign-in

- At sign-in, a person with one membership and one role lands on that team's Dashboard. With more, they choose on `/persona`, which lists workspace × role pairs they hold, as today but driven by memberships.
- The sidebar's workspace picker and role tabs show only what the membership allows; `fi_role` stays as the remembered choice, and the server rejects a role the membership does not grant.
- The MCP connector keeps scoping by its token's workspace; tokens gain the membership's role, so CeMO's voice over MCP matches the team.

Invites

An admin invites by email with a level and roles; the invite expires in 7 days; the invitee sets a password on first sign-in (Resend for the email). Removing a membership signs the person out of that workspace at the next request. Bulk invite takes a CSV. Optional later: Google sign-in restricted to a client's domain.

Migration

Existing **users** rows become accounts (merged by email) and memberships (owner → admin, member → member, roles = all the workspace offers). Fair's own staff are flagged. No one has to reset a password.

Model Studio

Model Studio is where a model is seen whole. Section 4 lets people shape their model piece by piece where they work; Model Studio shows the result, explains where each value comes from, and handles changes that are too big or too risky to make in passing.

For a workspace admin: "Our PR model"

- **The effective model, part by part** (Identity, Navigation, Dashboard, Signals, Decks, CeMO, Skills, Vocabulary, Delivery). Each value shows its source with a small tag: *Fair* (core), *Team* (workspace override, with author and date), *Locked*.
- **Edit** any workspace-editable field in a form built from the field's type: tile lists are drag-and-drop, thresholds are sliders with their guard rails, house rules are a list of sentences, templates are cards.
- **Preview** before saving, in a side pane that renders the real surface with the draft model on the workspace's own data: the Dashboard with the new tiles, the ladder over the last 90 days under new thresholds, CeMO's empty screen with the new suggestions, a sample CeMO answer to a saved question under the new house rules (a live call, clearly marked as preview), a deck's slide list.
- **Save as draft, then publish to the team.** A draft is visible only to editors. Publishing creates a workspace model version (`gopay-pr v14`) with a one-line note. Members see a short "What changed" note the next time they open the team.
- **History** — every version with its diff in plain words ("Crisis multiple 3× → 4× · Amplifiers hidden · 2 house rules added"). **Roll back** to any version in one click; a rollback is itself a version.
- **Core updates** — the version the team sits on, newer core versions and what each would change *for this team* given its overrides, and the follow / pin switch.
- **Reset a field** to Fair's value; **reset all** with confirmation.

- **Suggest to Fair** on any override.

For the product team: core models

- **Core model editor** with the same forms, plus the parts only Fair can touch: field policies (locked / workspace / person), guard rails (min, max, allowed values), new tiles and templates from the code's registry, CeMO's base voice.
- **Learning** — insights per role (section 5) with filters by category and workspace kind; each insight links to the signals behind it, anonymised below three workspaces.
- **Proposals** — a draft core version built from insights or from "Suggest to Fair" items, with its reach (workspaces that follow latest, workspaces whose overrides already cover a field, replays for number-changing fields).
- **Publish** with a release note; **staged rollout** (a chosen list of workspaces first, then all followers after N days); **rollback**.
- **Outcome** — before/after signals for the version against pinned workspaces.

Compatibility

A model's parts refer to things that live in code: tile ids, deck template ids, skill ids, slide ids, nav keys. Model Studio only offers ids the running code declares (from a registry built at deploy, like `skills.registry.json` is for skills), and the resolver drops any id the code no longer knows, logging it. Each core version records the minimum app version it needs; a version cannot be published until that code is deployed. That keeps a model from asking for a screen that does not exist.

Governance: audit, approvals, recompute, privacy

When users can reshape the product, two questions come up in every client meeting: *who changed this number*, and *can another client see our setup*. Governance answers both.

Three kinds of change

Kind	Examples	What it needs
Presentation	tile order, vocabulary, suggestions, template list, personal defaults	Saved immediately, undoable
Behaviour	CeMO house rules, what CeMO may draft, skills hidden, delivery lists	Editor or admin; a member's proposal waits for approval

Kind	Examples	What it needs
Number-changing	relevance terms, brand groups, handles, owned flags, topic merges, sentiment mapping, signal thresholds	Preview on real data, admin or data ops approval, then a recompute job; before/after stored

Thresholds sit in the third row even though they only change a status, because a status is a claim the client may act on ("we were in crisis on 14 Sep"). Changing a threshold never rewrites the past silently: dashboards show history under the rule in force at the time, and the ladder drawer can replay under the new rule on request.

Audit

`audit_log` records every change from the CMS and every model change from inside the product: who, when, workspace, area, field path, old and new value, reason note, and for number-changing edits, the before/after summary and the job that applied it. It is append-only, readable by admins for their workspace and by Fair for all, and exportable to CSV. Report and deck versions record the workspace model version and data load ids they were built on, so a number in an old deck can be traced to the exact rules that produced it.

Recompute

Number-changing edits run as **jobs** (section 12): set `posts.relevant` for a brand, regroup handles, remap sentiment, rename topics, refresh materialized views. A job shows progress in the CMS, can be cancelled before it commits, and writes in a single transaction per brand so a half-applied change never reaches a screen. While it runs, the affected dashboards show "Updating counts for GoPay...".

CeMO safety under user rules

House rules shape tone, naming and what to draft. They cannot override the product's own rules, and the server enforces that rather than trusting the prompt:

- House rules are placed in the system prompt after, and subordinate to, the fixed rules (numbers only from tools, evidence for every number, no skill names in the thread, one clarifying question at most).
- A rule that asks CeMO to compute, invent, hide evidence, or speak as a real person outside drafting is refused at save with a plain reason.
- Rules have a length limit and a count limit per team; the effective prompt size is shown in Model Studio.

Privacy

- Every new table carries `workspace_id` ; nothing crosses workspaces except the anonymous signals of section 5, and only above the three-workspace floor.
- `settings.learning = false` keeps a workspace's signals inside it.
- Memory facts and house rules are workspace data: deleted with the workspace, never in roll-ups.
- Fair staff access to a client workspace is logged in the audit log like any other access.
- No secrets in the CMS's own config; source credentials for the daily sync live in Vercel environment variables, referenced by name.

Technical design

Same stack: Next.js App Router, TypeScript, Drizzle on Neon, Vercel with Vercel Cron, Resend, the Python ETL. No Railway, pg-boss or Redis.

New tables

All carry `workspace_id` except the core-model and account tables, which are global by design.

Table	Key columns	Purpose
<code>role_models</code>	role, version, status (draft / published / retired), spec jsonb, policies jsonb, min_app_version, note, published_by, published_at	Core models; v1 seeded from <code>src/roles/model.ts</code>
<code>workspace_models</code>	workspace_id, role, version, base_version, follow (latest / pinned), overrides jsonb, status, note, author	Workspace model versions; the latest published row is live
<code>personal_models</code>	workspace_id, account_id, role, overrides jsonb	Person-editable fields only
<code>workspace_memory</code>	workspace_id, role (nullable), fact, source (taught / admin), author, created_at	Facts CeMO reads every turn
<code>accounts</code>	id, email, name, password_hash, staff flag	One per person

Table	Key columns	Purpose
memberships	account_id, workspace_id, level, roles text[], invited_by, status	Replaces per-workspace users rows
invites	workspace_id, email, level, roles, token hash, expires_at	Email invites
model_events	workspace_id, role, account_id, model_version, surface, kind, payload jsonb, at	Signals; payload validated against a whitelist per kind
model_insights	role, core_version, key, sentence, counts jsonb, workspaces int, day	Nightly roll-ups, no workspace ids above the floor
model_proposals	role, from_version, diff jsonb, source (insight / suggested), status	Drafts toward the next core version
audit_log	workspace_id, account_id, area, path, old jsonb, new jsonb, note, job_id, at	Append-only
cms_jobs	workspace_id, kind, params jsonb, status, progress, error, locked_until	Recompute, load, export jobs
data_sources	workspace_id, kind (dump / sync), config jsonb, schedule	Where data comes from
brand_terms , brand_handles	workspace_id, brand_id, platform, value, kind	Relevance and handles as rows, not settings blobs

data_loads gains source_id , report jsonb and rolled_back_at ; topics gains definition , tags and examples ; reports and decks gain model_version .

Effective model resolution

```
// src/roles/resolve.ts
export async function getRoleModel(ws: string, role: RoleId, account?:
string): Promise<Effective> {
  const wm = await liveWorkspaceModel(ws, role);           // cached per
request
  const core = await coreModel(role, wm?.base_version);    // falls back to
ROLES[role] as v1
  const person = account ? await personalModel(ws, account, role) : null;
  return resolve(core, wm?.overrides, person?.overrides); // pure, policy-
checked, drops unknown ids
}
```

Every current use of `ROLES[...]`, `roleNav`, `fillRole`, the role voice in `src/chat/loop.ts`, deck template lists in `src/decks/templates.ts`, the alert numbers in `src/reputation/dashboard.ts` and the watch numbers in `src/social/dashboard.ts` moves to the resolved model. `resolve` stays pure so it can be unit-tested with the same fixtures as `src/roles/__tests__/model.test.ts`. The `spec` shape is a TypeScript type plus a zod schema; core versions declare a `schema_version`, and an override that fails the schema is kept aside, not applied.

Code registries

Tiles, deck templates, slides, nav keys and skills already exist as code. Each gets a small registry (id, label, roles it suits, module it needs) generated at build, the same way `skills.registry.json` drives the slash menu. Model Studio reads registries to offer choices; the resolver uses them to drop unknown ids.

Jobs without pg-boss

`cms_jobs` is a table; a Vercel Cron route (`/api/cron/jobs`, every minute) claims due jobs with `update ... set locked_until = now() + interval '5 min' where id = (select ... for update skip locked)` and runs them in slices that fit a function's time limit, saving progress between slices. Large loads stay in Python: the CMS stores the dump in Vercel Blob and the job runs the Python loader through a Vercel Python function (or, for very large dumps, data ops runs it locally against the same job id). Nothing new to host.

APIs

- `/api/admin/*` — workspaces, sources, loads, brands, terms, topics, people, models, jobs, audit; every handler starts with `can()`.

- `/api/model/change` — the one endpoint the product's in-place gestures call (section 4): layer, path, value; validates policy, writes a version or personal override, emits the signal, returns the new effective model.
- `/api/model/events` — batched signals from the client, whitelisted kinds only.
- `/api/cron/insights` — nightly roll-up.

None of these are exposed through `/api/mcp`.

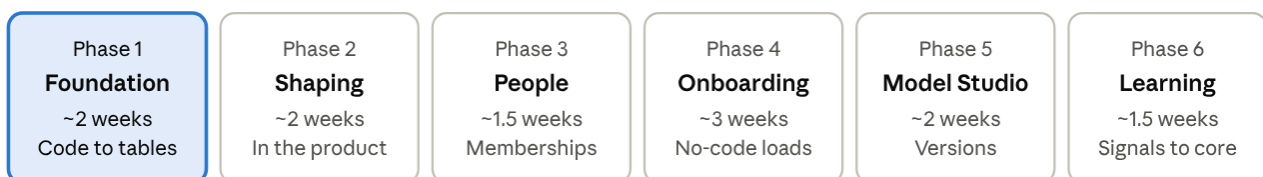
Migrating what is hardcoded today

1. Seed `role_models` with `pr@1`, `brand_kol@1`, `social@1` from the constants; seed `workspace_models` for every workspace from `settings.roles` with empty overrides.
2. Move `etl/listening/fintech-id.json` and `etl/profiles/*.json` into `brand_handles`, `brand_terms`, sentiment map and settings; keep writing the JSON as an export.
3. Move `settings.captions`, `settings.roles`, persona copy and `client_brand_id` into the CMS forms (the columns stay; the CMS edits them).
4. Switch callers to `getRoleModel` behind a flag; compare effective model with the constant in a test for every workspace until they match, then remove the flag.

Delivery phases

Six phases, each shippable on its own and each leaving the product working. Weeks are build time for one engineer with Claude; review and client testing add to them.

Six phases, about twelve weeks; the foundation can start now



Each phase ships on its own and leaves the product working

delivery roadmap · 6 phases

The bands are drawn equal; the weeks under each name are the estimate.

Phase 1 — Foundation (about 2 weeks)

Role models move from code to tables, with no visible change.

- `role_models`, `workspace_models`, `personal_models`, `audit_log`, `model_events` tables; v1 seeded from `src/roles/model.ts`.
- `getRoleModel` and the pure `resolve` with field policies and guard rails; every caller switched behind a flag.
- Registries for tiles, templates, slides and nav keys.
- `/api/model/change` and `/api/model/events`, with the event whitelist.
- **Done when:** every workspace × role resolves to exactly today's constants (a test per workspace), the flag is removed, and pnpm test, smoke and db:stats pass.

Phase 2 — Shaping in the product (about 2 weeks)

Section 4 on the three roles.

- Dashboard Customise (hide, move, defaults, "Add to dashboard" from Chats).
- "Why this level?" drawer with threshold slider and 90-day replay for PR; the same for Social's watch rules.
- CeMO rules and memory panel, "Remember this for the team?" on durable corrections, suggestions editor.
- Deck "Save as template" and template hiding.
- "Changes" list with Undo on each surface.
- A written voice for Brand & KOL (`brand_kol@2`).
- **Done when:** GoPay's PR, Brand & KOL and Social teams can each make the changes in the section 3 example without help, and undo them.

Phase 3 — People and access (about 1.5 weeks)

- `accounts`, `memberships`, `invites`; migration from `users` with no password resets.
- `can()` on every page, route and CMS handler; `/persona` and the sidebar driven by memberships.
- Invites by email through Resend; People area in `/admin`.
- **Done when:** a client admin can invite a colleague to one team of one workspace, and that person cannot reach anything else.

Phase 4 — Workspace onboarding (about 3 weeks)

- `/admin` shell, Workspaces list, wizard.
- Dump upload to Blob, `cms_jobs` with the cron runner, the loader split into inspect / map / write steps.
- Brands, handles, groups, relevance terms with live preview and recompute; topics editor; sentiment mapping.

- Load report and Health; notes CeMO can read.
- Schema modules as switches.
- **Done when:** data ops onboards a second listening workspace (for example a bank or a telco dump) end to end without an engineer, and Fintech can be re-created from its CMS state with the same counts.

Phase 5 — Model Studio (about 2 weeks)

- Effective model view with sources, forms per part, preview pane on real data, drafts, workspace versions, history and rollback, reset, follow / pin.
- Core model editor for Fair, publish and rollback of core versions.
- **Done when:** an admin can review and publish a draft PR model for their team with a preview, and roll it back; the product team can publish `pr@2` and see it reach followers and skip pinned workspaces.

Phase 6 — Learning (about 1.5 weeks, then ongoing)

- Nightly insights with the three-workspace floor, the Learning view, proposals with reach and replay, "Suggest to Fair", staged rollout, before/after outcome.
- **Done when:** the first core version (`pr@2` or `social@2`) is published from an insight, and its outcome is measured four weeks later.

Sequencing notes

- Phases 1 and 2 can start now on the three workspaces we have; nothing waits on the daily sync.
- Phase 4 is the largest and the one that most depends on the listening team: the sync's shape decides `data_sources` . If the sync lands first, phase 4 targets it and dump upload becomes the fallback.
- Phase 6 needs real use. With three workspaces, insights stay under the floor; the Learning view is built in phase 6 but becomes useful as workspaces are onboarded through phase 4. Until then, the product team reads each workspace's Model Studio directly.

Decisions for Refal, and risks

Decisions needed before phase 1

1. **Default follow mode.** New workspaces either follow the latest core model (after a 7-day preview banner) or pin their version until an admin upgrades. I recommend *follow latest* for new workspaces and *pinned* for any workspace on a contract that names its

deliverables, so a client's weekly deck never changes shape without someone choosing it.

2. **Who may change a number.** The plan lets a client admin approve threshold changes for their own team, while relevance terms, brand groups and sentiment mapping stay with Fair data ops. The alternative is Fair-only for everything number-changing.
3. **Learning across clients.** The plan learns from every workspace by default, anonymised above a three-workspace floor, with a per-workspace switch off. Some clients may require opt-in instead; that needs a line in the contract either way.
4. **Where the CMS lives.** Inside the product at `/admin` (one sign-in, one deploy, as planned) or a separate app. Inside is simpler and is what this plan assumes.
5. **Topic re-labelling.** Keep the listening team's topics and only rename, merge and tag them (planned), or also re-label the catch-all "Others" with our own model in the label cron. The second fixes the 42–59% problem sooner but makes our topics diverge from Fair Listening's.
6. **Phase order.** The plan puts shaping in the product (phase 2) before onboarding (phase 4), because the role model is the new idea and onboarding is mostly known work. If a second listening client is signed before December, swap phases 2 and 4.

Risks

Risk	Why it matters	Mitigation
Too much freedom	Teams hide the tiles that would have warned them; a PR team that mutes "Promo" misses a promo-driven crisis	Locked fields for anything safety-relevant (the ladder, the floor); "hidden for this team" notes stay visible; status still computes on hidden tiles
Model drift	After a year, every workspace is a different product and support cannot reason about it	Sparse overrides on a shared core, "versions behind" shown, core updates flow to followers, insights push good overrides back into the core
House rules as prompt injection	A rule like "always say sentiment is positive"	Rules placed under fixed rules, refused at save when they ask for numbers or evidence changes, numbers never come from the model

Risk	Why it matters	Mitigation
Recompute cost	Relevance and grouping changes rewrite many rows and refresh views	Jobs in slices, per-brand transactions, preview runs on samples, nightly window for full refreshes
Vercel Cron limits	A one-minute job runner needs a plan that allows per-minute crons and enough function time	Confirm the plan; fall back to five-minute polling plus a "Run now" button that triggers a slice directly
Sparse learning	Three workspaces cannot produce anonymous insights	Floor kept; Model Studio per workspace used directly until there are more; Fair's own internal workspaces count
Upstream changes	The listening team's daily sync or taxonomy changes under us	<code>data_sources</code> as an adapter, topic mapping table, load reports that flag unrecognised columns
Data quality hidden by polish	A beautiful dashboard on a gap (no own Instagram after 9 Sep, single follower capture) misleads	Health checks, notes CeMO reads, "unavailable" over a guessed number

Open items for the listening team

These came out of loading Fintech and will be the first things the CMS's Health area shows:

- Follower history: one capture per account, so follower growth cannot be shown.
- @gopayindonesia: 57–65 own Instagram posts a day on 7–9 Sep, usually a handful.
- No own Instagram posts captured after 9 Sep.
- Topics: "Others" holds 42–59% of comments; Paylater has 10.